

Opportunity Radar

What we're building, why it's built this way, and how it all fits together.

The problem

A student can find a scholarship online and still be stuck with three questions: can I actually apply, what am I missing, and how much should I trust this. Opportunity Radar brings all three into one workflow instead of leaving the student to figure them out alone.

1. Find real opportunities

The Actor scrapes opportunity listings from supported sources and turns them into structured data: title, organization, link, deadline, description, and eligibility information.

The first version starts with one source. The goal is to prove the Actor can reliably collect real opportunity data before any of the intelligence layers get added on top.

2. Check the hard requirements

The student provides a profile: education level, field of study, country, GPA or grade, and whether they need funding.

We will not simply hand everything to an LLM and ask if the student is eligible. Requirements that can be objectively checked get handled with plain code instead: is the required degree level satisfied, does the GPA meet the threshold, is the student's country eligible, has the deadline passed. These checks are deterministic. If something clearly fails, the opportunity gets rejected or flagged without spending an LLM call on it.

3. Use the LLM only where language needs interpreting

Some requirements are hard to turn into a simple rule. A line like "preference will be given to applicants with demonstrated research experience" needs judgment, not a

threshold check. That is what the LLM is for.

The result keeps three things visibly separate: hard requirements checked by code, ambiguous criteria interpreted by the LLM, and anything still missing or unclear. Keeping these apart makes the reasoning easy to explain and much harder to accuse of just hallucinating an eligibility decision.

4. Assess trust risk with real evidence

We will not tell a student "AI has verified this scholarship." That claims more certainty than any evidence can back up. Instead the system assesses risk.

For each opportunity, it collects external evidence through Google Custom Search and checks signals such as: whether upfront payment is required, whether eligibility information is vague or absent, whether there's disproportionate urgency, whether the sponsoring organization has an independent presence online, and whether the opportunity shows up anywhere else.

The output is a risk level, always shown with its reasons:

High Risk

Requires upfront payment.

No independent web presence found for the sponsoring organization.

These are observed signals, not a claim that something has been proven fraudulent.

5. Test it properly, not just for the demo

We'll build a small evaluation set: real opportunities plus a handful of deliberately constructed adversarial listings, each one clearly labeled as a **synthetic test case** with known risk signals built in, so we can check whether the system actually catches them.

We'll also run the system against roughly ten real opportunities, so the demo isn't resting entirely on examples we made up ourselves.

6. Ship it as an Apify Actor

Apify is the cloud platform this whole thing runs on, and it's also the reason the hackathon exists in this shape. An Actor is just a program packaged to run in Apify's cloud: it takes structured input, does its work in an isolated container, and returns structured output that anyone can trigger through the web UI, an API call, or a schedule. Apify handles the parts that are normally painful in scraping work: rotating proxies so we don't get blocked, running headless browsers at scale, retrying failed requests, and storing results in a queryable dataset.

For us, everything from the scraper in phase 1 through the trust scoring in phase 3 lives inside this one Actor. It's not a website with a scraper bolted on, the Actor is the product. Apify also has a public Store where finished Actors can be published so other people can run them, paying per use, which is where the "earn" half of the hackathon theme actually comes from: a working Actor can keep generating value after judging ends, not just during the demo.

It takes a student's profile and search parameters, processes real listings, and returns structured results:

```
{ "opportunity": "PhD Scholarship, University of X", "deadline": "2026-10-14", "source": "phdportal",  
  "hardRequirementsMet": true, "eligibilityMatch": "Partial", "missingRequirements": ["Research experience not confirmed"],  
  "llmInterpretation": "Preference criterion, not mandatory", "trustRisk": "Some Concerns",  
  "trustEvidence": ["Requires a processing fee", "Found on two independent sources"] }
```

Structured output like this is what makes the Actor reusable by other tools and developers through Apify, not just a one-off demo.

The full workflow

Student profile ↓ Opportunity sources ↓ Apify Actor / crawler ↓ Structured listings ↓ Hard requirement checks (deterministic) ↓ Ambiguous requirements → LLM interpretation ↓ External trust evidence → risk assessment ↓ Transparent opportunity result

How we're building it

Layer by layer. Each phase gets built and tested before the next one starts, so we always know what's actually working rather than hoping it all comes together at the end.

- | | |
|---------|---|
| PHASE 1 | Scraper pulls real listings from one source. In progress. |
| PHASE 2 | Eligibility matching: deterministic checks plus LLM interpretation for ambiguous language. |
| PHASE 3 | Trust scoring backed by real web search evidence. |
| PHASE 4 | A second data source, and a simple frontend so a student can actually use this: a form for their profile and a results page showing eligibility and trust for each opportunity. |
| PHASE 5 | The evaluation set (real plus labeled synthetic listings), then demo rehearsal. |

The frontend piece in phase 4

The form fields match the input schema already defined: education level, field of study, country, whether funding is needed, and grade (optional). The results view calls the Actor through the Apify API and shows each opportunity with its eligibility status and trust level, so it's just sending the form data and rendering what comes back. No need to touch the scraping or matching code.

This only happens once the backend is actually working end to end. A working Actor with no UI is worth more than a nice UI wrapping something broken.

Still open: which LLM

We haven't locked this in yet. Claude and Gemini are both on the table, and the honest deciding factor is who already has working API access to which one, since that removes setup time we don't have much of. The project scaffold currently assumes Claude as a placeholder, not a final decision.

The one line to remember

The scraper is infrastructure. The LLM is one reasoning component. The product is the combination: discovery, eligibility checking, and transparent trust assessment, all in one place, so a student can see what they're dealing with before they invest their time or money.

Opportunity Radar, built for She Code Africa x Apify BuildHER Hackathon 2026. Repo:
github.com/LovableLynx/opportunity-radar (private, ask to be added).